DSA

TASK-2

Title : Code trace challenge - think, trace predict and verify

Topic : Stack operations using an array

Code understanding

(1) Data Structure used.

The program uses a stack implemented using an array.

→ A stack follows the LIFO

Ex :

- push 10, 20, 30
- pop removes 30 first
- The remaining stack contains 10 and 20

(2) Variables and intial values

Variable Size	Intial Value	purposese. Maximum stack capacity
Stack (size)	5 All Elements initially un used	Stores stack elements

top	-1	Indicates the top element
item	Not initialized initially	Stores the value passed to push()
i	Initialized inside display() loop	Traverses the stack elements

(3) functions used

function	purpose
push(int item)	Inserts an element into the stack
pop()	Remove and return the top element
display()	Displays all elements from bottom to top.
main()	calles the Stack function in sequence.

(4) conditions and Loops

if (top == size - 1) checks whether the Stack is full

if (top == -1) check whether the stack is empty

for (i=0; i<= top ; i++) prints each elements from index 0 to top.

indicate the top element
ores the ue passed to bc
ases the ements

purpose :

push

(1) check the stack is full

(2) If full → overflow

(3) Increment top

pop

(1). check the stack is empty

(2) If empty - under flow.

(3) Decrement top.

Manual code tracing.

The stack size is 5, so the valical array indices are 0, 1, 2, 3, 4.

Step	Operation	Item	top	Stack elements
	Initial State	-	-1	Empty
1	push (10)	10	0	10
2	push (20)	20	1	10, 20
3	push (30)	30	20	10, 20, 30
4	display ( )	-	2	10, 20, 30
5	pop ( )	30	1	10, 20
6	push (40)	40	2	10, 20, 40
7	push (50)	50	3	10, 20, 40, 50
8	push (60)	60	4	10, 20, 40, 50, 60
9	push (70)	70	4	10, 20, ~~30~~ 40, ~~50~~ 60
10	display ( )	-	4	10, 20, 40, 50, 60
11	pop ( )	60	3	10, 20, 40, 50
12	pop ( )	50	2	10, 20, 40, 50
13	pop ( )	40	1	10, 20, 40
14				

14	pop()	40	1	10,20
15	pop()	20	0	10
16	pop()	10	-1	empty
17	display	0	-1	empty
18	pop()	-1	-1	Empty

Value Tracing and function return values. The pop() function returns the top element and decreases top by one.

pop	operation	top before	return value	top after
first	pop	2	30	1
second	pop	4	60	3
third	pop	3	50	2
fourth	pop	2	40	1
fifth	pop	1	20	0
sixth	pop	0	10	-1
seventh	pop	-1	-1	-1

loop variable i during display()

first display top = 2

i	printed value
0	10
1	20
2	30.

Stack is Empty
Stack underflow

popped :- 1

final Stack State:

top = -1

Stack contents empty

overflow : occurs during push (70)

underflow :- occurs during the final pop()

Reason : the Stack reaches its maximum capacity of five elements.

What is Analysis

Stage	original problem	modified program
push operation	push (70) attempted	push(70) removed.
Result	Stack overflow	no overflow
Stack contents	10, 20, 40, 50, 60	10, 20, 40, 50, 60
final Stage	Empty	empty

Reason :

The original program can't intent to because the Stack is Full. Removing the statement eliminate the overflow message.

	printed values
1	
0	10
1	20
2	40
3	50
4	60.

When top = -1, the display function prints stack is empty and the loop does no execute.

output prediction.

pushed : 10
pushed : 20
pushed : 30
stack : 10 20 30
pop : 30
pushed : 40
pushed : 50
pushed : 60

stack overflow

stack : 10 20 40 50 60
popped : 60
popped : 50
popped : 40
popped : 20
popped : 10.

Analysis:-

The program correctly handles stack overflow and underflow. However returning -1, underflow can be confusing if -1 is also a valid stack element.

Collection:

Return a states value such as a far failure, For success it pass the popped value through pointer

Verification & Test cases:-

Verification

The program is executed using a C compiler after completing the manual trace. The predicted output is compared with the actual to check corrections.

Test case 1

```
push (25);
item = pop();
printf ("popped : %d\n", item)
```

Test case 2

```
item = pop();
printf("popped : %d\n", item);
```

Test case 3:-

```
push(1);
push(2);
push(3);
push(4);
push(5);
push(6);
display();
```

Test case	Expected output	Actual output
Test 1	25 popped Stack empty	pushed: 25 popped: 25 Stack is empty
Test 2	underflow return -1	Stack underflow -1 save.
Test 3	Over flow : Five elements	

Conclusion :

The program demostration the implementation of Stack array in C. It used the LIFO principle push() inserts an element. pop() removes the top elements and display() function points the Stack contents. The program successfully detects Stacks Over flow and underflow.